

server-state 설계·운영 매뉴얼

역할: 서버별 원하는 상태를 정의하고, 실제 소유 모듈의 점검·복구 경로를 dry-run 계획으로 연결한다.

1. 왜 별도 조정 계층이 필요한가

GPU 서버 한 대에는 Docker, NVIDIA driver/toolkit, Kubernetes node, Kerberos NFS, exporter와 사용자 container 전제조건이 함께 필요하다. 그러나 이 기능을 하나의 거대한 playbook에 넣으면 소유권, 위험도와 변경 주기가 사라진다.

server-state 는 다음 두 질문만 담당하는 얇은 desired-state 계층이다.

- 1. 이 서버에 무엇이 있어야 하는가?
- 2. 점검 또는 복구는 어느 모듈의 어떤 명령이 소유하는가?

현재 구현은 의도적으로 dry-run 중심이다. 로컬 inventory와 profile을 읽고 remote check와 remediation command를 출력하지만 production host에 접속하거나 변경하지 않는다. apply --execute 는 구현되지 않았고 명시적으로 거부된다.

2. 디렉터리 지도

경로	핵심 기능
bin/server-state	Python CLI wrapper
server_state/cli.py	list-hosts , check , plan , apply 출력
server_state/inventory.py	공용 JSONL inventory 로드와 host 선택
server_state/profiles.py	YAML profile/catalog 파싱, set 확장과 template render
config/profiles.yml	desired-state check/remediation의 권위 있는 정의
ansible_playbook/bootstrap_gpu_server.yml	공통 GPU host baseline의 tag 기반 playbook
ansible_playbook/kerberos_nfs_client_recovery.yml	기존 host GSS readiness/recovery 순서
docs/module-boundaries.md	모듈별 소유권 경계
docs/standard-gpu-server-pipeline.md	신규/기존 서버 표준 단계
tests/	inventory와 profile expansion unit test

3. 데이터 모델

3.1 서버 inventory

기본 inventory는 `user-lifecycle/server_info/servers.jsonl` 이다. `server-state` 가 별도 서버 목록을 만들지 않는 이유는 주소, SSH port와 논리 server ID의 중복 권위를 피하기 위해서다.

loader는 FARM/LAB host 이름과 JSONL 필드를 정규화하고 다음 선택 방식을 제공한다.

- `all`
- `farm`, `lab` domain
- `farm8`, `FARM8` 같은 개별 host/server ID
- comma 또는 공백으로 구분한 여러 selector

3.2 profile catalog

`config/profiles.yml` 은 profile과 profile set을 정의한다. 각 profile은 소유 모듈, read-only check와 remediation을 가진다.

check kind:

kind	동작
<code>local-inventory</code>	inventory 존재를 로컬에서 확인
<code>local-path</code>	저장소 또는 controller 경로 존재 확인
<code>template-value</code>	서버별 render 값 존재 확인
<code>remote-read</code>	실행하지 않고 읽기 전용 원격 명령을 표시

remediation mode:

mode	출력
<code>command</code>	<code>DRY-RUN</code> 명령과 safety level
<code>manual</code>	<code>MANUAL</code> 상태와 소유 runbook/reference

profile set은 여러 profile을 정해진 순서로 확장한다. 신규 서버와 기존 서버가 같은 작은 profile을 재사용하므로 전역 설정이 한쪽 pipeline에서 빠지는 것을 줄인다.

4. 표준 GPU 서버 pipeline

순서	profile	소유자	목적
1	<code>baseline-host</code>	server-state	inventory, SSH, sudo, hostname preflight

순서	profile	소유자	목적
2	os-common	server-state	apt, NFS, Kerberos, network 도구
3	docker-engine	server-state	Docker와 systemd cgroup driver
4	nvidia-driver	server-state	host driver와 package hold
5	nvidia-container-runtime	server-state	Docker/containerd NVIDIA toolkit
6	kubernetes-node	server-state	kubeadm/kubelet/kubectl와 join gate
7	network-tuning	server-state	storage NIC RX queue 영속화
8	kerberos-nfs-client	kerberos-nfs	realm, keytab, GSS와 mount profile
9	monitoring-exporters	monitoring	cluster/GPU user exporter
10	user-container-host	user-lifecycle	lifecycle 실행 전제조건

new-host-bootstrap 와 existing-host-drift 가 이 profile들을 각각 신규 설치와 기존 상태 점검 관점에서 조합한다. managed-host 는 기존 관리 서버의 기본 alias다.

5. 모듈 소유권

server-state가 직접 소유하는 것

- 공통 OS package와 repository
- Docker engine의 systemd cgroup 설정
- NVIDIA driver/toolkit 설치 형태
- Kubernetes node package 전제조건
- storage NIC RX queue tuning
- profile/host 선택과 상태 vocabulary

다른 모듈에 위임하는 것

영역	실제 소유자	server-state의 행동
exporter와 dashboard	monitoring	배포 playbook 명령을 계획
NAS/AD/Kerberos 정책	kerberos-nfs	manual/gated runbook 참조
사용자·컨테이너 transaction	user-lifecycle	inventory를 읽고 CLI/playbook을 계획
WOL·boot sequence	remote-operations	controller 설정과 service install 점검
DECS image	container-images	필요 profile에서 image test를 참조

조정 계층이 다른 모듈의 구현을 복제하지 않는 이유는 한쪽 수정이 다른 복사본에 반영되지 않는 drift를 막고, 고위험 작업의 승인 gate를 유지하기 위해서다.

6. CLI 사용법

host 목록:

```
cd /home/jy/server_manage
./server-state/bin/server-state list-hosts --hosts all
./server-state/bin/server-state --format json list-hosts --hosts farm
```

점검 계획:

```
./server-state/bin/server-state check \
  --hosts farm8 \
  --profile managed-host
```

복구 계획:

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile monitoring-exporters
```

신규 서버 전체 순서:

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile new-host-bootstrap
```

현재 **apply** 도 같은 dry-run plan만 출력한다.

```
./server-state/bin/server-state apply \
  --hosts farm8 \
  --profile existing-host-drift
```

자동화가 결과를 소비할 때는 **--format json** 을 사용한다. text 출력 parsing에 의존하지 않기 위해 구조화 출력을 별도로 제공한다.

7. 안전 모델

상태 vocabulary는 다음 의미를 가진다.

상태	의미
OK	로컬에서 확인 가능한 조건 충족
MISSING	로컬 파일/값 없음
DRY-RUN	실행할 원격 check 또는 remediation 표시
MANUAL	자동화하지 않은 고위험/도메인 소유 작업
UNKNOWN	지원하지 않는 kind/mode

`bootstrap_gpu_server.yml`에는 실제 task가 있지만 CLI가 자동 실행하지 않는다. 운영자는 생성된 Ansible 명령의 host limit, tag와 extra variable을 검토하고 우선 `--check --diff`로 실행한다.

Kubernetes join은 token이 짧게 유효하고 잘못된 join이 scheduler와 GPU workload에 영향을 주므로 자동 기본 값이 없다. Kerberos host keytab 준비도 도메인마다 Samba/winbind 또는 SSSD/adcli 흐름이 달라 hook으로 남기고 secret 출력을 `no_log`로 숨긴다.

8. Kerberos NFS recovery 순서

기존 host에서 mount부터 실행하지 않는다. 다음 순서를 지킨다.

```
keytab 존재
-> machine kinit -k
-> NFS service kvno
-> rpc_pipefs
-> rpc-gssd active
-> mount 존재/응답 확인
-> 필요한 경우에만 gated recovery
```

읽기/계획 확인:

```
ansible-playbook server-state/ansible_playbook/kerberos_nfs_client_recovery.yml \
--limit farm8 \
--check --diff \
-e server_state_hosts=farm8
```

모든 사전조건을 확인하고 share가 실제로 missing일 때만 mount action을 명시한다.

```
ansible-playbook server-state/ansible_playbook/kerberos_nfs_client_recovery.yml \
--limit farm8 \
-e server_state_hosts=farm8 \
-e server_state_recover_mount_now=true
```

9. 새 profile 추가

1. 변경의 구현 소유 모듈을 먼저 정한다.
2. `config/profiles.yml` 에 작은 profile을 추가한다.
3. 부작용 없는 관측만 `checks` 에 둔다.
4. remediation은 가능하면 `--check --diff` 명령으로 표현한다.
5. 공유 서비스, key, join처럼 위험한 작업은 `manual` 과 reference로 남긴다.
6. 신규/기존 서버 모두 필요한 항목이면 두 profile set의 올바른 위치에 넣는다.
7. host template expansion과 set ordering unit test를 추가한다.
8. 실제 소유 모듈의 문서와 이 매뉴얼의 pipeline 표를 갱신한다.

10. 테스트

```
cd /home/jy/server_manage/server-state
python3 -m unittest discover -s tests -v
```

변경 후 추가 확인:

```
./bin/server-state --format json check \
--hosts farm8 \
--profile existing-host-drift

./bin/server-state plan \
--hosts lab10 \
--profile new-host-bootstrap
```

실제 서버 변경이 없어도 command rendering, owner, safety와 host별 경로가 맞는지 검토할 수 있어야 한다.

11. 향후 실행 모드 원칙

실제 `apply --execute` 를 추가하려면 단순히 subprocess 호출을 허용해서는 안 된다. 최소한 다음 조건이 필요하다.

- host와 profile별 명시적 승인

- check 결과와 remediation 사이의 상태 재검증
- owner module별 실행 adapter
- timeout, audit log와 secret redaction
- partial failure와 재실행의 idempotency
- manual safety 항목의 자동 실행 금지

이 조건이 설계·검토되기 전까지 dry-run 전용 상태가 안전한 기본값이다.